

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom-up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:5

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

*I Enamala Gowtham (192311190) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
Enamala Gowtham

Annexure A

DESIGN TASK

Q1. Consider the grammar:

$S \rightarrow ABD$

$A \rightarrow a \mid Db \mid \epsilon$

$B \rightarrow gD \mid dA \mid \epsilon D$

$\rightarrow e \mid f$

A compiler development team is developing a table-driven predictive parser for a domain-specific language. Before implementation, the team must determine whether the grammar is suitable for LL(1) parsing and design all necessary parsing components.

Tasks

(i) Construct FIRST and FOLLOW Sets

Compute the FIRST and FOLLOW sets for all non-terminals (S, A, B, and D). Clearly show the steps involved in the computation.

(ii) Construct the Predictive Parsing Table

Using the computed FIRST and FOLLOW sets, design the LL(1) predictive parsing table for the grammar.

(iii) Verify the LL(1) Property

Analyze the predictive parsing table and determine whether the grammar satisfies the LL(1) condition.

Your analysis should include:

FIRST/FIRST conflict analysis

FIRST/FOLLOW conflict analysis for ϵ -productions

Identification of conflicting or multiple entries in the parsing table

Final conclusion on whether the grammar is LL(1)

(iv) Demonstrate Predictive Parsing

Using the constructed parsing table, perform predictive parsing for a valid input string derived from the grammar.

Your demonstration should include:

Stack contents at each step

Remaining input

Parsing action performed

Final acceptance or rejection of the input string

(v) Provide Design Recommendations

Based on your analysis:

State whether the grammar can be directly used for LL(1) parsing.

If the grammar is not LL(1), identify the cause of the conflict and suggest suitable grammar modifications.

Discuss the advantages of using a predictive parser for syntax analysis. (10 Marks)

Q2. Consider the grammar:

$E \rightarrow E + E \mid E - E$

$E \rightarrow E * E \mid E / E$

$E \rightarrow (E)$

$E \rightarrow \text{id}$

A compiler development team is implementing a stack-based shift-reduce parser for arithmetic expressions. To validate the parser design, the team must analyze the parsing process for the following input string:

id * id / (id + id)

Your Design Should:

(i). Construct the stack implementation of the shift-reduce parser for the given grammar.

(ii). Demonstrate the complete parsing process by showing all shift and reduce operations performed on the input string.

(iii). Identify the handle recognized at each reduction step and specify the corresponding production used.

(iv). Explain the parser's decisions during parsing and discuss any shift-reduce conflicts that may arise, along with the role of operator precedence and associativity in resolving them.

(v). Present a detailed parsing trace showing:

Stack contents

Remaining input

Parsing action (Shift / Reduce / Accept)

(vi). Verify whether the input string is successfully accepted by the parser. (10 Marks)

Q3.Design and implement a recursive descent parser for the following grammar:

$$S \rightarrow (L) \mid a$$
$$L \rightarrow S L'$$
$$L' \rightarrow , S L' \mid \epsilon$$

Further, demonstrate the parsing process for the input string:

(a)

Your design should:

- (i). Develop recursive procedures for each non-terminal (S, L, and L').
- (ii). Apply top-down parsing without backtracking using a predictive recursive descent approach.
- (iii). Clearly show the sequence of function calls and terminal-symbol matching during parsing.
- (iv). Determine whether the input string is accepted or rejected, with appropriate justification.
- (v). Incorporate a simple error detection and recovery mechanism to handle invalid input strings gracefully. (10 Marks)

Q4.Design an operator precedence parser for arithmetic expressions involving operators +, -, * and parentheses ().

Further, demonstrate parsing for the input string:

id - id * id

Your design should:

- (i). Construct an unambiguous grammar suitable for operator precedence parsing
- (ii). Define precedence relations ($<\cdot$, $=\cdot$, $\cdot>$) among terminals
- (iii). Develop the operator precedence table
- (iv). Implement stack-based parsing algorithm
- (v). Show step-by-step shift and reduce actions with precedence handling (10 Marks)

Q5.Design and Construct an LALR(1) Parser for the Following Grammar

Consider the grammar:

$$S \rightarrow A A$$
$$A \rightarrow aA \mid b$$

A compiler development team is designing an LALR(1) parser for a simple language construct. Before implementing the parser, the team must verify the grammar, construct the required parsing components, and validate the parser using a sample input string.

Demonstrate the operation of the parser for the input string:

bb

Your Design Should:

- (i). Construct the augmented grammar for the given grammar.
- (ii). Develop the canonical collection of LR(1) item sets, including closure and goto computations.
- (iii). Compute the FIRST and FOLLOW sets required for parser construction.
- (iv). Merge LR(1) states having identical cores and construct the LALR(1) parsing table (ACTION and GOTO).
- (v). Demonstrate the parsing process for the input string aabb by showing:

Stack contents

Input buffer

Parsing actions (Shift, Reduce, Accept)

Production used during each reduction

- (vi). Verify whether the input string is accepted by the parser and justify the result. (10 Marks)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation

Q1. LL(1) predictive parser Design

Given Grammar:

$$S \rightarrow A B D$$

$$A \rightarrow a | D b | \epsilon$$

$$B \rightarrow g D | d A | \epsilon$$

$$D \rightarrow e | f$$

(i) construct FIRST and Follow sets

Step 1: FIRST sets

FIRST(D)

$$D \rightarrow e | f$$

$$\text{FIRST}(D) = \{e, f\}$$

FIRST(A)

$$A \rightarrow a$$

$$A \rightarrow D b$$

$$A \rightarrow \epsilon$$

$$\text{Since } \text{FIRST}(D) = \{e, f\}$$

$$\text{FIRST}(D b) = \{e, f\}$$

Therefore,

$$\text{FIRST}(A) = \{a, e, f, \epsilon\}$$

FIRST(B)

$$B \rightarrow g D$$

$$B \rightarrow d A$$

$$B \rightarrow \epsilon$$

hence

$$\text{FIRST}(B) = \{g, d, \epsilon\}$$

FIRST(S)

$$S \rightarrow A B D$$

FIRST(A)

$$= \{g, e, f, \epsilon\}$$

Since ϵ belongs to FIRST(A), include FIRST(B)

Therefore,

$$\text{FIRST}(S) = \{a, e, f, g, d\}$$

FIRST Table

Non-Terminal

S

A

B

D

FIRST set

$$\{a, d, e, f, g\}$$

$$\{a, e, f, \epsilon\}$$

$$\{g, d, \epsilon\}$$

$$\{e, f\}$$

Step 2: follow sets

start symbol

$$\text{Follow}(S) = \{\$ \}$$

from

$$S \rightarrow A B D$$

for A

$$\text{Follow}(A) = \text{FIRST}(BD)$$

FIRST(B)

$$= \{g, d, \epsilon\}$$

Since ϵ exists,

Include FIRST(D)

$$= \{e, f\}$$

Hence

$$\text{Follow}(A) = \{g, d, e, f\}$$

For B

After B comes D

$$\text{Follow}(B) = \text{FIRST}(D)$$

hence

$$\text{Follow}(B) = \{e, f\}$$

For D

D is the last symbol

$$\text{Follow}(D) = \text{Follow}(S)$$

$$= \{\$ \}$$

Final FOLLOW(D)

{ ϵ , b, e, f}

from

$B \rightarrow dA$

A occurs at end

$\text{FOLLOW}(A) = \text{FOLLOW}(B)$

$\text{FOLLOW}(B)$

$= \{e, f\}$

Already present

FOLLOW Table

Non-Terminal

Follow set

S

$\{\epsilon\}$

A

$\{g, d, e, f\}$

B

$\{e, f\}$

D

$\{\epsilon, b, e, f\}$

(ii) Construct the Predictive parsing Table

Non-Terminal

a

b

d

e

f

g

ϵ

S

$S \rightarrow ABD$

-

$S \rightarrow ABD$

$S \rightarrow ABD$

$S \rightarrow ABD$

$S \rightarrow ABD$

-

A

$A \rightarrow a$

-

~~$A \rightarrow \epsilon$~~

$A \rightarrow db$

$A \rightarrow db$

$A \rightarrow \epsilon$

-

B

-

-

$B \rightarrow dA$

$B \rightarrow \epsilon$

$B \rightarrow \epsilon$

$B \rightarrow gD$

-

D

-

-

-

$D \rightarrow e$

$D \rightarrow f$

-

-

Demonstrate Predictive parsing

Choose a valid input string

a g e e

Derivation

S

$\Rightarrow ABD$

$\Rightarrow aBD$

$\Rightarrow agDD$

$\Rightarrow ageD$

$\rightarrow agee$

Input
agee\$

Stack	Input	Action
\$	agee\$	$S \rightarrow ABD$
\$DBA	agee\$	$A \rightarrow a$
\$DBa	agee\$	Match a
\$DB	gee\$	$B \rightarrow gD$
\$DDg	gee\$	Match g
\$DD	eed	$D \rightarrow e$
\$De	ee\$	Match e
\$D	e\$	$D \rightarrow e$
\$e	e\$	Match e
\$	\$	Accept

Result : The input string agee is accepted.

Q₂ : Shift-Reduce parser Design

Given Grammar

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

$$E \rightarrow E * E$$

$$E \rightarrow E / E$$

$$E \rightarrow (E)$$

$$E \rightarrow id$$

Input string :

$$id * id / (id + id)$$

(i) Stack implementation of shift — Reduce parser

A shift - Reduce parser uses a stack to recognize handles and reduce them according to grammar productions.

Algorithm.

1. Initialize the stack with $\$$
2. Read the input from left to right
3. Shift symbols from the input onto the stack
4. when the top of the stack matches a handle, Reduce it using the appropriate production
5. Repeat until the stack contains $\$E$ and the input is $\$$
6. Accept the input if parsing completes successfully.

(ii) Complete parsing process

input

$id * id / (id + id) \$$

Step	Stack	Remaining input	Action
1	$\$$	$id + id / (id + id) \$$	Shift id
2	$\$ id$	$* id (id + id) \$$	Reduce $E \rightarrow id$
3	$\$ E$	$+ id / (id + id) \$$	Shift $+$
4	$\$ E +$	$id / (id + id) \$$	Shift id
5	$\$ E * id$	$/ (id + id) \$$	Reduce $E \rightarrow id$
6	$\$ E * E$	$/ (id + id) \$$	Reduce $E \rightarrow E * E$
7	$\$ E$	$/ (id + id) \$$	Shift $/$
8	$\$ E /$	$(id + id) \$$	Shift $($
9	$\$ E / ($	$id + id) \$$	Shift id
10	$\$ E / (id$	$+ id) \$$	Reduce $E \rightarrow id$
11	$\$ E / (E$	$+ id) \$$	Shift $+$
12	$\$ E / (E +$	$id) \$$	Shift id
13	$\$ E / (E + id$	$) \$$	Shift $)$
14	$\$ E / (E + id$	$) \$$	Shift $)$
15	$\$ E / (E + id$	$) \$$	Accept

(iii) Handle Recognition		production used
Reduction step	Handle	
1	id	$E \rightarrow id$
2	id	$E \rightarrow id$
3	$E * E$	$E \rightarrow E * E$
4	id	$E \rightarrow id$
5	id	$E \rightarrow id$
6	$E + E$	$E \rightarrow E + E$
7	(E)	$E \rightarrow (E)$
8	E / E	$E \rightarrow E / E$

(iv) parser decision and shift reduce conflicts

Parser decision

The parser shifts symbols until a valid handle appears

When a handle matches a right hand side of a production, it performs a Reduce operation

The process continues until the entire input is

reduced to the start symbol E .

(iv) verification

The parser successfully reduces the input string

$id + id / (id + id)$

to the start symbol E using valid shift and reduce operations.

Q3: Recursive Descent parser Design

Given Grammar:

$$S \rightarrow (L) \mid a$$
$$L \rightarrow SL'$$
$$L' \rightarrow , SL' \mid \epsilon$$

Input string:

(a)

(i) Recursive procedures for each Non-Terminal

Algorithm

Procedure S()

S()

{

if (nextToken == '(')

{

match '(';

L();

match (')');

}

else if (nextToken == 'a')

{

match ('a');

}

else

{

error();

}

}

Procedure L()

L()

{

S();

Lprime();

Procedure Lprime()

Lprime()

{ if (nextToken == "'')

{ match(' ');

S();

Lprime();

}

else {

{

/* E-production */

return;

}

}

Procedure match()

match(x)

{

if (nextToken == x)

advance();

else :

error();

}

(iii) Sequence of Function calls and Terminal Matching

Input

(a) \$

Step	Function call	Current input	Action
1	S()	(a) \$	calls
2	match(' ')	(a) \$	match(' ')
3	L()	a) \$	call L
4	S()	a) \$	call S

5	match('a')	a) \$	match 'a'
6	Lprime()	) \$	Apply $L' \rightarrow \epsilon$
7	match(')')	) \$	match ')'
8	return	\$	Parsing complete
9	Accept	\$	Input Accepted

(iv) Acceptance of Input String

Input

(a)

Derivation

S

$\Rightarrow (L)$

$\Rightarrow (SL')$

$\Rightarrow (aL')$

$\Rightarrow (a)$

The entire input is Consumed Successfully, and the parser Reaches the end Marker \$

Result

The input string (a) is Accepted.

(v) Error Detection and Recovery Mechanism

Error Detection

The parser detects an error whenever the current input symbol does not match the expected terminal,

Q4: Operator precedence parser design

(i) Construct an unambiguous Grammar

The following grammar respects operator precedence and associativity:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid id$$

Operator precedence

1. parentheses () - Highest precedence
2. Multiplication * and Division /
3. Addition + and Subtraction -

(ii) Precedence Relation

Symbol pair

Relation

$\$ < . id$

Shift

$\$ < . ($

Shift

$id . > +$

Reduce

$id . > -$

Reduce

$id . > *$

Reduce

$id . > /$

Reduce

$id . >)$

Reduce

$id . > \$$

$+ < . id$

Shift

$- < . id$

Shift

$/ < . ($

Shift

(iii) Operator precedence Table

	id	+	-	*	/	(	)	\$
id	-	>	>	>	>	-	>	>
+	<	>	>	<	<	<	>	>
-	<	>	>	>	<	>	>	>
*	<	>	>	>	<	>	>	>
/	<	>	>	>	<	>	>	>
(	<	<	<	>	<	<	=	=
)	-	>	>	<	>	-	>	>
\$	<	<	<	<	<	<	-	Accept

(iv) Stack parsing Algorithm

Algorithm

1. Initialize the stack with \$
2. Read the next input symbol
3. Compare the precedence of the stack top termin with the next input symbol,
4. If the Relation is $<$ or $=$, perform shift
5. If the Relation is $>$ or $>$, perform Reduce.

(v) Verification

Input string

id - id + id

final stack
\$ ∈

Q5: Design & Construct and LALR(1) parser

Given Grammar:

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

Input string for parsing
aabb

(i) Augmented Grammar

Introduce a new start symbol S'

$$(0) S' \rightarrow S$$

$$(1) S \rightarrow AA$$

$$(2) A \rightarrow aA$$

$$(3) A \rightarrow b$$

(i) Augmented Grammar

Introduce a new start symbol S'

$$(0) S' \rightarrow S$$

$$(1) S \rightarrow AA$$

$$(2) A \rightarrow aA$$

$$(3) A \rightarrow b$$

(ii) Canonical Collection of LR(1) Item sets

$$I_0 = \text{Closure}(\{S' \rightarrow \cdot S, \$\})$$

$$S' \rightarrow \cdot S, \$$$

$$S \rightarrow \cdot AA, \$$$

$$A \rightarrow \cdot aA, \{a, b\}$$

$$A \rightarrow \cdot b, \{a, b\}$$

Transition

$$\text{Goto}(I_0, S) = I_1$$

$$\text{Goto}(I_0, A) = I_2$$

$$\text{Goto}(I_0, a) = I_3$$

$$\text{Goto}(I_0, b) = I_4$$

I_1

$$S' \rightarrow S \cdot, \$$$

Accept state

I_2

$$S \rightarrow A \cdot A, \$$$

$$A \rightarrow \cdot a A, \$$$

$$A \rightarrow \cdot b, \$$$

Transitioning

$$\text{Goto}(I_2, A) = I_5$$

$$\text{Goto}(I_2, a) = I_3$$

$$\text{Goto}(I_2, b) = I_4$$

I_3

$$A \rightarrow a \cdot A$$

$$A \rightarrow \cdot a A$$

$$A \rightarrow \cdot b$$

Transitioning

$$\text{Goto}(I_3, A) = I_6$$

$$\text{Goto}(I_3, a) = I_3$$

$$\text{Goto}(I_3, b) = I_4$$

I_4

$$A \rightarrow b \cdot$$

Reduce string

$$S \rightarrow AA.$$

Reduce using

$$S \rightarrow AA$$

I₆

$$A \rightarrow aA.$$

Reduce using

$$A \rightarrow aA$$

(iii) FIRST and FOLLOW sets

FIRST sets

Non - Terminal

A

S

FIRST

{a, b}

{a, b}

FOLLOW sets

Since S is the start symbol,

$$\text{FOLLOW}(S) = \{\$ \}$$

From

$$S \rightarrow AA$$

GOTO Table

State

S

A

0

1

2

2

5

3

6

Productions

$R_1 : S \rightarrow AA$

$R_2 : A \rightarrow aA$

$R_3 : A \rightarrow b$

(v) parsing process for input string

Input

aabb\$

Step	Stack	Input Buffer	Action
1	0	aabb\$	Shift a
2	0a3	abb\$	Shift a
3	0a3a3	bb\$	Shift b
4	0a3ab4	b\$	Reduce $A \rightarrow b$
5	0a3a3A6	b\$	Reduce $A \rightarrow aA$
6	0a3A6	b\$	Reduce $A \rightarrow aA$
7	0A2	b\$	Shift b
8	0A2b4	\$	Reduce $A \rightarrow b$
9	0A2A5	\$	Reduce $S \rightarrow AA$
10	0S1	\$	Accept

Productions used during Reduction

Step	Production
1	$A \rightarrow b$
2	$A \rightarrow aA$
3	$A \rightarrow aA$
4	$A \rightarrow b$
5	$S \rightarrow AA$

(vi) verification

Input string

aabb

Derivation

S

$\Rightarrow AA$

$\Rightarrow aAA$

$\Rightarrow aaAA$

$\Rightarrow aabA$

$\Rightarrow aabb$

The parser successfully reaches the augmented

start production

$S' \rightarrow S^*$

Therefore, The input string is accepted by LR(1)

parser.